

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2º Semestre

Revisão

Inserir Dados

```
db.collectionName.insertOne()  
db.collectionName.insertMany()
```

Obter Dados:

```
db.collectionName.find()  
db.collectionName.findOne()
```

```
db.users.insertOne(  ← collection  
  {  
    name: "sue",      ← field: value  
    age: 26,          ← field: value  
    status: "pending" ← field: value  } document  
  }  
)
```

```
db.users.find(  ← collection  
  { age: { $gt: 18 } }, ← query criteria  
  { name: 1, address: 1 } ← projection  
) .limit(5) ← cursor modifier
```

Revisão

Obter Dados:

Documentos com preço igual a 5 e uma quantidade superior a 1:

```
db.collectionName.find({ $and: [ {preco: 5}, {quantidade: {$gt: 1}} ]})
```

Documentos com preço não superior a 5

```
db.collectionName.find({ preco: {$not: {$gt: 5}} })
```

Revisão

Atualizar Dados:

```
db.collectionName.updateOne()  
db.collectionName.updateMany()
```

```
db.users.updateMany(  
  { age: { $lt: 18 } },  
  { $set: { status: "reject" } }  
)
```

← collection
← update filter
← update action

Remover Dados:

```
db.collectionName.deleteOne()  
db.collectionName.deleteMany()
```

```
db.users.deleteMany(  
  { status: "reject" }  
)
```

← collection
← delete filter

Revisão

Count method:

```
> db.client.find().count() , número de documentos encontrados na query  
> db.client.find({ age: 20 }).countDocuments()
```

Limit method:

```
> db.client.find().limit(<number>) , põe limite no número de resultados retornados  
> db.client.find({ age: 20 }).limit(1)
```

Sort method:

```
> db.client.find().sort({first_name: 1})  
ordena os documentos (1 for ascending order and -1 for descending order)
```

Group multiple methods

```
> db.client.find().sort({first_name: 1}).limit(5)
```

Skip method:

```
> db.client.find().skip(10).limit(10)
```

Exercícios

1. Qual a ocupação do *user* “Clifford Johnathan”? Apresentar apenas *name* e *occupation* na resposta.
2. Quantos utilizadores existem com idades compreendidas entre 18 e 30 anos?
3. Quais são as 10 escritoras mais velhas?
4. Criar um novo *user* na base de dados com: *name*, *gender* and *occupation*.
5. Remover o utilizador criado no ponto 4.
6. Mudar o campo “*occupation: programmer*” para “*developer*”.
7. Quantos filmes foram lançados entre 1984 e 1992?
8. Encontrar todos os filmes que têm como género “Musical” e “Romance”.

Exercícios

```
use('CloudProject');

// 1
db.Users.find(
  {name: 'Clifford Johnathan'},
  {name: 1, occupation: 1}
)

// 2
db.Users.find({
  $and: [
    {age: {$gte: 18}},
    {age: {$lte: 30}}
  ]
}).count()

//3
db.Users.find(
  {gender: 'F', occupation: 'writer'},
  {_id: 0, name: 1, age: 1}
).sort({age: -1}).limit(10)

//4
db.Users.insertOne(
  {
    name: 'John Doe',
    age: 20,
    gender: 'M',
    occupation: 'technician/engineer'
  }
)
```

```
//5
db.Users.deleteOne({
  name: 'John Doe',
  gender: 'M',
  occupation: 'technician/engineer'
})

//6
db.Users.updateMany(
  {occupation: 'programmer'},
  {$set: {occupation: 'developer'}}
)

//7
db.movies.find(
  {title: {$regex: /198[4-9]|199[0-2]/}}
).count()

//8
db.movies.find({
  $and: [
    {genres: {$regex: /musical/i}},
    {genres: {$regex: /romance/i}}
  ]
}).count()
```

Operador \$regex

Usado para pesquisa de um *padrão* ou sequência de caracteres em uma string.

```
> db.products.find( { sku: { $regex: /789/ } } )
```

Atributo acaba com a string 789: { sku: { \$regex: /789\$/ } }

Atributo começa com a string 789: { sku: { \$regex: /^789/ } }

Procurar documentos onde o atributo 'sku' começa com 'ABC' mas é case-insensitive:
 { sku: { \$regex: /^ABC/i } }



Framework WSGI - Web Server Gateway Interface. Interface Python between applications and servers.

Implementação de *web applications* de forma eficaz, escaláveis e de fácil manutenção.

Permite receber dados do utilizador e enviar dados de volta dependendo do tipo de *request*

Python

Flask - *install e setup*

- Criar virtual environment:

Virtual environment, gestão de dependências do projeto.

Permite instalar bibliotecas específicas ao nosso projeto (pré-instalado com python 3.3)

```
> mkdir backend-app  
> cd backend-app  
> python3 -m venv venv
```

- Activar virtual environment:

Mac/Linux:

```
> source venv/bin/activate
```

Windows:

```
> venv\Scripts\activate
```

Flask - *install e setup*

Instalar Flask

> python3 -m pip install flask

- Criar ficheiro “app.py”
- Run:
 - > python app.py

```
from flask import Flask

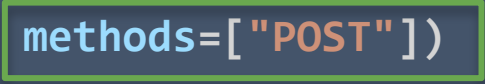
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, World!"

if __name__ == '__main__':
    app.run(debug=True)
```

Flask - *methods*

```
@app.route("/books", methods=["POST"])
def new_book():
    #adicionar novo livro
    return "POST REQUEST"
```



Define o método HTTP do *Route*:
(POST, GET, DELETE, PUT)

methods=["POST", "GET"]

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/books', methods=['GET', 'POST'])
def books():
    if request.method == 'POST':
        # (...)
        return "POST"
    else:
        # (...)
        return "GET"
```

Flask - *URL variables*

```
@app.route('/user/<username>')
def show_user_profile(username):
    # (...)
    return f'User {username}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # (...)
    return f'Post {post_id}'
```

string	(default) any text without a slash
int	Accepts positive integers
float	accepts positive floating point values
path	like string but also accepts slashes

Flask - URL parameters

http://host.com/api/v1/movies?type=comedy&year=2000

```
@app.route('/movies', methods=['POST'])
def get_query_example():

    type = request.args.get('type')
    year = request.args.get('year')

    return (...)
```

Flask + MongoDB

PyMongo: Interface MongoDB.

- Permite estabelecer conexões com servidores MongoDB
- Manipulação de dados MongoDB: insert / find / update / delete

```
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = MongoClient(<MONGO_URL_STRING>,
                    tls=True,
                    tlsAllowInvalidCertificates=True)
db = client[<DATABASE_NAME>]
```

<https://pymongo.readthedocs.io/en/stable/index.html>

Instalar PyMongo:

```
> python3 -m pip install pymongo
```

Flask + MongoDB

```
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = (...)
db = client[<DATABASE_NAME>]

@app.route("/users", methods=["GET"])
def get_users():
    users = db.users.find().limit(10)
    cursor = list(users)
    return jsonify(cursor)
```

```
> collection.insert_one({"key": "value"})
> collection.insert_many([
    {"key1": "value1"},
    {"key2": "value2"}
])

> cursor = collection.find({"key": {"$gt": 5}})

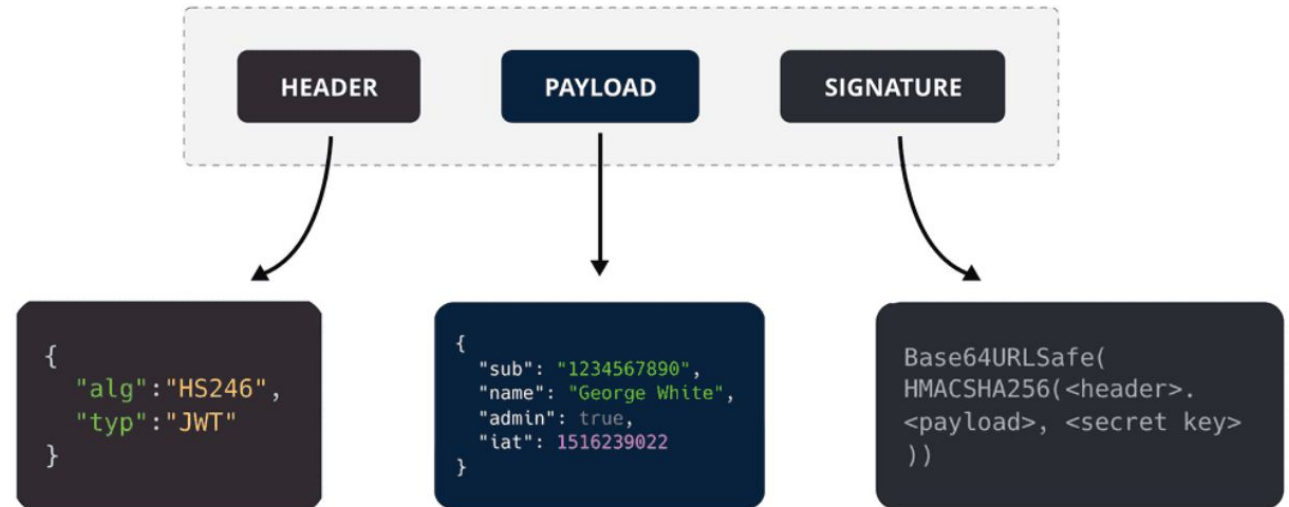
> cursor = collection.find().sort("key", pymongo.ASCENDING)
# ou DESCENDING

> collection.update_one(
    {"key": "value"},
    {"$set": {"new_key": "new_value"}}
)
> collection.update_many(
    {"key": "value"},
    {"$set": {"new_key": "new_value"}}
)

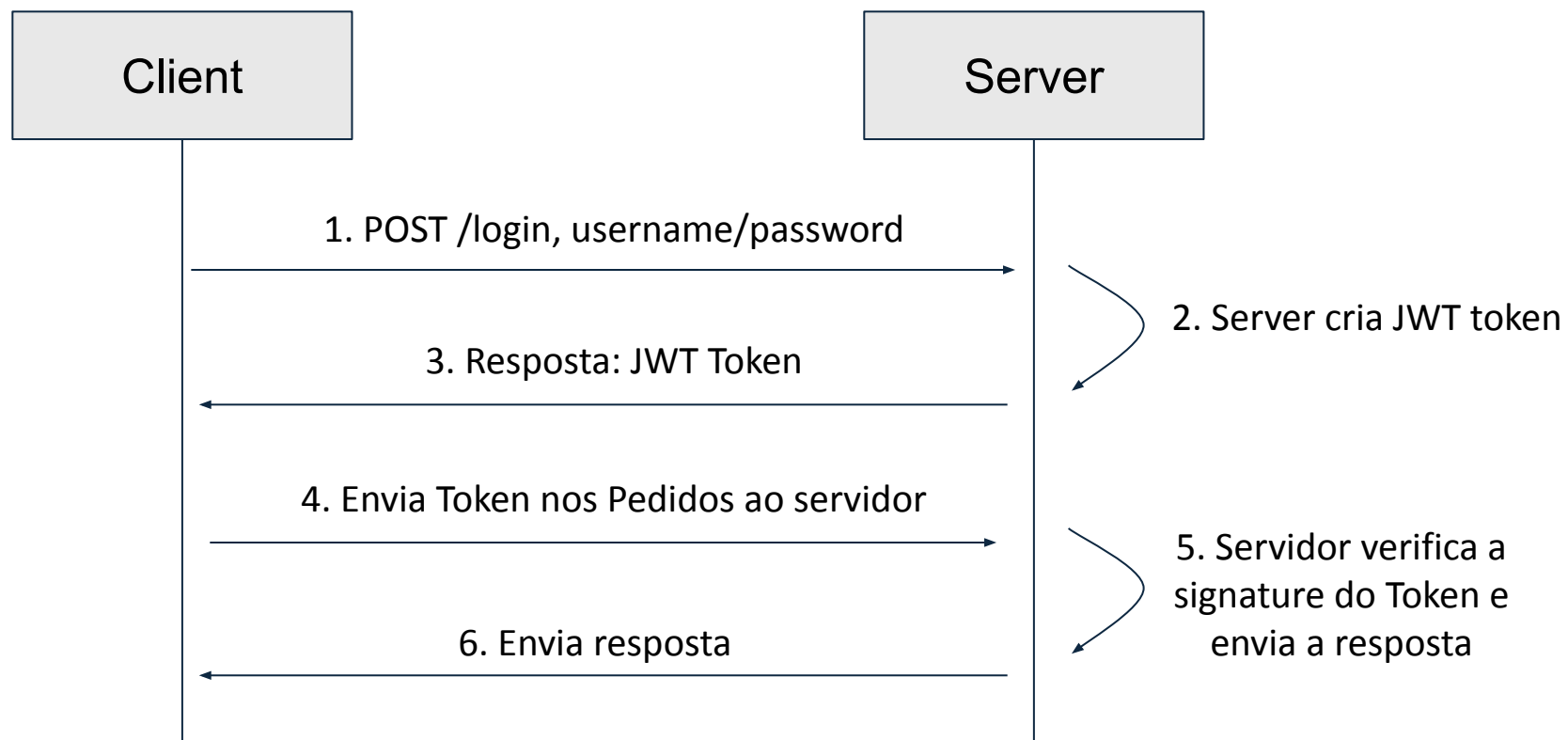
count = collection.count_documents({"key": "value"})
```


JWT: JSON Web Token

- Usado para autenticação e transmissão de dados de forma segura, geralmente cliente-servidor
- Ficheiro JSON *encoded*
- JWT é gerado pelo servidor após método de autenticação
- Constituído por 3 partes:
 - Header: tipo de token e o algoritmo de criptografia usado
 - Payload: Informações a enviar / tempo de expiração do token
 - Signature: garante a integridade do token. String gerada pelo header, payload e a chave secreta da App



JWT: JSON Web Token



JWT: JSON Web Token

Criar Token

```
@app.route('/user/login', methods=['POST'])
def login():
    dados = request.json
    # Fazer Login do user
    # Check se user é válido e se tem o campo confirmation = True
    # Apenas gerar Authentication Token se confirmation = True

    token = jwt.encode({
        'username': dados['username'],
        'exp': datetime.utcnow() + timedelta(minutes=1)
    }, app.config['SECRET_KEY'], algorithm="HS256")

    return jsonify({'token': token}), 200
```

Instalar JWT

```
> pip install PyJWT
```

Import JWT

```
import jwt
from datetime import datetime,
timedelta
```

JWT: JSON Web Token

SECRET_KEY

```
(...)
app = Flask(__name__)
CORS(app)

app.config['SECRET_KEY'] = 'SEGREDO'

(...)
```

- JWT secret key: chave única da aplicação usada para assinar e verificar tokens JWT
- Usada no processo de criação e validação
- Usada para calcular a assinatura digital e anexada ao token

```
> node -e
"console.log(require('crypto').randomBytes(32).toString('hex'))"
```

JWT: JSON Web Token

```
import jwt
from datetime import datetime, timedelta
from functools import wraps
from bson import ObjectId

app.config['SECRET_KEY'] = 'SEGREDO'

def token_required(func):
    @wraps(func)
    def decorated(*args, **kwargs):
        token = request.args.get('token')
        if not token:
            return jsonify({'message': 'Token is missing!'}), 401

        try:
            data = jwt.decode(token, app.config["SECRET_KEY"],
                                algorithms=["HS256"])

            except jwt.ExpiredSignatureError:
                return jsonify({'expirado': True}), 401

            return func(*args, **kwargs)

        return decorated
```

Função *decorated*, vai servir de middleware para as diferentes *routes*

Object data:

```
data = jwt.decode(...)
print(data["username"])
```

JWT: JSON Web Token

```
@app.route("/protected/route", methods=["GET"])
@token_required
def get_user_X():
    users = db.users.find({"name": "Barry Erin"})
    cursor = list(users)
    return jsonify(cursor)
```

Depois de definir a função `token_required`, podemos “proteger” os routes que precisam de autenticação: **@token_required**